

Реализованные подходы к хранению файловой системы

Все три реализации используют общий интерфейс `IFileSystem`. Он задаёт операции чтения и записи файла (`op_read`, `op_write`), создания каталога (`op_mkdir`), вывода непосредственных потомков (`op_ls`), перемещения файла или поддерева (`op_mv`), поиска по поддереву (`op_find`) и оценки занятой памяти (`get_memory_usage`). Пути абсолютные; реализации проверяют некорректные компоненты и не допускают перемещение каталога внутрь собственного поддерева.

Поиск по шаблону реализован как `glob`-сопоставление имени узла: `*` означает произвольную последовательность символов, а `?` — один символ. Поэтому сложность `find` зависит не только от структуры данных, но и от числа посещённых узлов.

Подход А: наивное N-арное дерево

Подход А реализован классом `TreeFileSystem`. Файловая система хранится в виде дерева: каждый `Node` содержит имя, признак каталога, данные файла, указатель на родителя и вектор владеющих указателей на дочерние узлы. Корень хранится непосредственно в объекте файловой системы.

Чтобы найти путь, реализация разбирает его на компоненты и на каждом уровне линейно просматривает вектор детей. Это делает представление простым и компактным: каталог естественно содержит только своих потомков, `ls` возвращает их без обхода остальных узлов, а перенос поддерева не требует менять пути потомков. Операция `mv` извлекает один `unique_ptr` из старого вектора детей, меняет имя и родителя узла и вставляет его в новый вектор.

Главный недостаток подхода — отсутствие индекса. При росте ширины каталогов любое обращение по пути последовательно выполняет линейный поиск среди детей. Таким образом, этот вариант хорошо подходит как базовый, прозрачный по структуре подход, но не оптимизирует адресный доступ к произвольному абсолютному пути.

Подход В: дерево с индексом полных путей

Подход В реализован классом `FileSystemB`. Основная структура остаётся деревом с теми же связями «родитель–дети», однако дополнительно поддерживается `unordered_map<string, Node*>`: абсолютному пути сопоставляется адрес узла в дереве. Благодаря этому чтение, запись, создание каталога и проверка существования пути используют прямое обращение к хэш-индексу вместо последовательного обхода уровней дерева.

Дерево сохраняет преимущества для структурных операций: `ls` проходит только по вектору детей, а `find` обходит только указанное поддерево. Цена оптимизации — дополнительная память на ключи-пути, бакеты хэш-таблицы и указатели, а также более сложная операция `mv`. При переносе узла необходимо удалить из индекса старые пути всего переносимого поддерева, изменить связи в дереве, затем заново построить и добавить новые абсолютные пути потомков. Поэтому быстрый доступ по одному пути достигается за счёт дорогой синхронизации двух представлений при массовом перемещении.

Подход С: плоская хэш-таблица путей

Подход С реализован классом `FlatHashFileSystem`. В нём нет явного дерева: единственный контейнер `unordered_map<string, Node>` хранит запись для каждого абсолютного пути. Значение содержит только признак каталога и данные файла; имя, родительская связь и положение в иерархии выводятся из строкового ключа. Иерархия проверяется по префиксам путей.

Такое представление обеспечивает ожидаемо быстрое обращение к конкретному пути и не дублирует дерево указателями и векторами детей. Однако информация о детях и поддеревьях не материализована. Поэтому `ls` проходит по всей хэш-таблице и выбирает записи, у которых заданный каталог является непосредственным родителем; `find` также начинает с полного прохода и фильтрации по префиксу. При `mv` нужно найти все пути переносимого поддерева, отсортировать их по длине и заменить старый префикс на новый в каждом ключе. Следовательно, вариант С выгоден для точечных обращений, но хуже для операций, зависящих от структуры каталога.

Сравнение асимптотик

Для таблицы введём обозначения: N — общее число узлов; H — глубина пути; W — максимальное число детей одного каталога; K — число непосредственных детей каталога; S — число узлов в просматриваемом или переносимом поддереве; M — число узлов в переносимом поддереве; L — длина пути; B — размер передаваемых или возвращаемых данных файла; G — стоимость сопоставления одного имени с glob-шаблоном.

В таблице для В и С указана ожидаемая сложность хэш-операций. Хэширование и сравнение строк учитываются через L ; при допущении ограниченной длины пути этот множитель обычно считают константным. Худший случай для `unordered_map` может быть линейным из-за коллизий.

Операция	А: N-арное дерево	В: дерево + индекс путей	С: плоская хэш-таблица
<code>read</code>	$O(H \cdot W + B)$	$O(L + B)$	$O(L + B)$
<code>write</code>	$O(H \cdot W + B)$	$O(L + B)$	$O(L + B)$
<code>mkdir</code>	$O(H \cdot W)$	$O(L)$	$O(L)$
<code>ls</code>	$O(K)$	$O(K)$ после индексного доступа	$O(N \cdot L)$; без учёта строк — $O(N)$
<code>find</code>	$O(S \cdot G)$	$O(S \cdot G)$	$O(N \cdot L + S \cdot G)$
<code>mv</code>	$O(H \cdot W)$; потомки не переименовываются	$O(M \cdot (H + L))$: переиндексация поддерева	$O(N \cdot L + M \cdot \log M + M \cdot L)$: поиск, сортировка и замена ключей

Асимптотики показывают основной компромисс. А экономит память и делает структурные операции естественными, но платит линейным поиском в каждом каталоге. В ускоряет адресные операции индексом, сохраняя эффективные обходы дерева, но использует больше памяти и переносит стоимость в `mv`. С хранит минимум структурных связей и быстро находит известный путь, однако лишается локального доступа к детям: `ls`, `find` и перемещение каталога требуют просмотра заметной части либо всей таблицы.

Память

У всех подходов есть неизбежная стоимость данных файлов. В А дополнительно хранятся имена узлов, указатели на родителей и ёмкости векторов детей. В хранит те же данные дерева и ещё одну копию абсолютных путей в хэш-индексе, поэтому его накладные расходы наиболее высоки. С не хранит указатели родителя и векторы детей, но ключами хэш-таблицы являются полные пути; следовательно, стоимость строк равна суммарной длине всех абсолютных путей. Практический расход памяти также зависит от коэффициента загрузки `unordered_map`, числа бакетов и избыточной ёмкости `vector` и `string`.

Таким образом, выбор реализации зависит от профиля нагрузки: А является простым базовым вариантом, В ориентирован на частые точечные обращения по известному пути, а

С целесообразен, когда преобладают точечные операции и редки обходы каталогов, поиск и перенос больших поддеревьев.

Обоснование выбора сетки

Сетка параметров подбиралась так, чтобы покрыть как типичные, так и граничные режимы работы in-методу файловой системы, но при этом оставить число запусков практически выполнимым. В основной регулярной части используются значения глубины (D), ширины (W) и заполнения (F):

$$D \in \{2, 5, 10\} \quad W \in \{5, 10, 15\} \quad F \in \{0.3, 0.6, 0.95\}$$

Такая сетка даёт три характерных уровня для каждого измерения: малый, средний и большой. Этого достаточно, чтобы увидеть не только локальные колебания, но и общую тенденцию роста стоимости операций при усложнении структуры дерева.

Выбор узлов 2, 5, 10 по глубине и 5, 10, 15 по ширине объясняется тем, что они отражают качественно разные формы дерева. $D = 2$ описывает почти плоскую иерархию, $D = 5$ соответствует умеренно вложенной структуре, а $D = 10$ уже моделирует глубокие каталожные цепочки, где начинают проявляться накладные расходы на проход по пути. Аналогично, $W = 5$ соответствует узкому дереву, $W = 10$ является промежуточным вариантом, а $W = 15$ позволяет увидеть эффект широких директорий, где возрастают затраты на поиск дочернего узла и обход содержимого. Значения $F = 0.3, 0.6$ и 0.95 покрывают разреженный, средний и почти полностью заполненный режимы.

Помимо регулярной части, в коде добавлены corner-case конфигурации: глубокие и узкие деревья (10, 2), (15, 2), (20, 2), (15, 3) и широкие, но неглубокие деревья (2, 30), (2, 50), (3, 30), (3, 50). Это важно, потому что именно на таких формах сильнее всего расходятся свойства реализаций: одни структуры данных лучше ведут себя на длинных путях, другие на широких каталогах. Следовательно, одной только симметричной регулярной сетки было бы недостаточно.

Логарифмическая шкала при визуализации результатов оправдана тем, что стоимость операций и потребление памяти растут неравномерно и в ряде режимов различаются на порядки. Например, переход от умеренной структуры к очень крупной может резко увеличить число узлов и тем самым сделать различия между реализациями плохо различимыми на линейной шкале. Логарифмическая шкала позволяет одновременно видеть и малые, и большие значения, не теряя информации о поведении на “лёгких” конфигурациях и не сжимая их в почти горизонтальную линию.

Отдельно важна оценка времени экспериментов. Для одной пары “файловая система + профиль нагрузки” регулярная сетка уже даёт $3 \times 3 \times 3 \times 3 = 81$ точку, а с добавлением граничных shape-профилей число конфигураций становится ещё больше. При параметрах $repeats = 2$ и $ops = 2000$ один job на практике занимает минуты, а полный набор запусков по нескольким профилям и нескольким реализациям даёт десятки минут wall-clock времени даже при распараллеливании. Иными словами, в текущей постановке мы уже слегка упираемся в производительность доступного железа: дальнейшее механическое уплотнение сетки сразу увеличивает время прогона и требования к ресурсам. Поэтому выбранная конфигурация является рабочим компромиссом между полнотой покрытия и вычислительной стоимостью. При этом само исследование на этом не заканчивается: его можно продолжать и расширять по мере появления доступа к более производительной машине, при более агрессивном распараллеливании или более узко сформулированных гипотезах, для которых имеет смысл локально сгущать сетку только в интересующих областях параметров.

Комбинаторный взрыв и необходимость аппроксимации

Проблема комбинаторного взрыва возникает из-за того, что итоговое число рассматриваемых конфигураций растёт как произведение числа вариантов по каждому измерению. Даже если оставить только основные параметры, получаем произведение по глубине, ширине, заполнению, профилю распределения путей, профилю операций, типу файловой системы, числу повторов и числу операций в прогоне. При добавлении новых уровней хотя бы по одному измерению объём пространства состояний растёт мультипликативно, а не линейно.

В данной задаче полный перебор особенно дорог по двум причинам. Во-первых, каждая точка сетки требует не просто арифметического расчёта, а полноценной генерации синтетической файловой системы, подготовки последовательности операций и последующего прогона на конкретной реализации. Во-вторых, если цель состоит не только в сравнении нескольких заранее заданных конфигураций, но и в рекомендации на произвольных пользовательских параметрах, то заранее промерить все возможные комбинации D , W , F , локальности, распределения и вероятностей операций вообще невозможно: пространство становится слишком большим, а многие параметры по сути непрерывные.

Именно поэтому аппроксимация является единственным практичным путём. Вместо полного перебора всех комбинаций строится суррогатная модель, которая по входным параметрам быстро оценивает ожидаемые метрики: среднюю задержку, хвостовую задержку, память и пропускную способность. Такая модель не обязана идеально воспроизводить поведение системы в каждой точке; её задача в другом: достаточно точно сохранять относительный порядок альтернатив и давать ответ за миллисекунды, а не за минуты или часы реального бенчмарка.

Практический смысл аппроксимации состоит в том, что она переводит задачу из режима “дорогое измерение каждой новой точки” в режим “дешёвый прогноз с выборочной верификацией”. Сначала реальные бенчмарки выполняются на репрезентативной сетке и дают опорные данные. Затем на их основе или на основе инженерных эвристик строится приближённая функция, которая оценивает поведение системы между измеренными точками. После этого пользователь может получать рекомендацию для произвольного набора параметров без запуска полного набора экспериментов.

Таким образом, комбинаторный взрыв здесь не является абстрактной теоретической проблемой, а напрямую ограничивает возможность полного перебора в разумное время. Аппроксимация в такой постановке не просто удобное улучшение, а необходимый инструмент, который делает систему рекомендаций и анализ результатов практически применимыми.

Оценка времени проведения эксперимента

Оценка времени проведения эксперимента была сделана на основе двух источников: анализа кода раннера и одного пробного запуска в Release-сборке для модели C на профиле refactoring. Выбор именно модели C можно обусловить предположением о ее наибольшей (среди других моделей) нагрузке при некоторых конкретных операциях: в частности, `or_ls` просматривает все элементы `nodes_` при сборе дочерних узлов, `or_find` проходит по всей хеш-таблице, а `or_mv` предварительно собирает все пути в перемещаемом поддереве. Поэтому модель C выглядит как одна из наиболее медленных на крупных конфигурациях. Профиль refactoring был выбран как наиболее близкий к сбалансированному из доступных: в нём присутствуют все шесть операций, а вероятности распределены без нулевых компонент.

В эксперименте рассматриваются 7 профилей нагрузки и 3 реализации файловой системы. При этом каждый отдельный запуск для фиксированной пары “одна реализация + один профиль” внутри себя всё равно оказывается довольно объёмным: раннер проходит по 17 share-конфигурациям дерева, а для каждой из них перебирает ещё 3 значения заполнения и 3 профиля распределения путей. Иначе говоря, один такой запуск включает $17 \times 3 \times 3 = 153$ отдельных конфигурации, поэтому его время уже естественно измерять не секундами, а минутами.

Пробный запуск в Release-режиме для модели C на профиле refactoring использовался как заведомо тяжёлый ориентир. Идея здесь в том, что это не “средний” и не “оптимистичный” случай, а скорее близкая к верхней границе оценка времени на один профиль для одной реализации. Поэтому, если даже в таком консервативном сценарии полный эксперимент оценивается в часы, а не в дни, то требование практической выполнимости можно считать выполненным с запасом. Иными словами, исследование уже достаточно тяжёлое, чтобы учитывать ограничения железа, но при этом всё ещё реалистично для запуска на обычной машине, особенно при распараллеливании.

Если экстраполировать эту оценку на полное исследование, то для трёх реализаций и шести профилей последовательно потребовалось бы порядка 18 jobs, то есть примерно 6-9 часов чистого wall-clock времени. При распараллеливании по 4 процессам ожидаемое время уменьшается примерно до 1.5-2.5 часов, однако уже в этом режиме становится заметно влияние производительности доступного железа, кэшей и общей нагрузки на память. Именно поэтому текущая сетка выглядит практически достижимой, но её дальнейшее расширение при текущей реализации моделей и бенчмарка уже требует либо более мощной машины, либо более выборочного сгущения только в наиболее интересных областях параметров.

Анализ выбранного метода аппроксимации

Более подробный анализ по мл моделям находится в `ml_model/research.pdf`

На основе анализа специфики задачи для проведения экспериментов были выбраны два метода: линейная регрессия с L2-регуляризацией (Ridge) в качестве baseline-модели и ансамбль решающих деревьев (Random Forest) в качестве целевой нелинейной модели.

Обоснование выбора:

- **Ожидаемая форма зависимостей**
 - **Нелинейность:** В подходе А время поиска зависит от глубины D и ширины W как $O(D \cdot \log W)$ или $O(D \cdot W)$. Это явно нелинейная зависимость. Чтобы линейная регрессия могла её уловить, мы применим логарифмирование признаков
 - **Экспоненциальный рост:** В подходах В и С групповые операции вроде `ls` или `find` зависят от размера поддерева, который при увеличении коэффициента заполнения F может расти экспоненциально
- **Требования к интерпретируемости**
 - **Линейная регрессия** даёт максимальную интерпретируемость: знак и величина коэффициента перед признаком показывают, насколько сильно и в какую сторону меняется метрика при изменении признака
 - **Random Forest** менее прозрачен, но он предоставляет оценку важности признаков. С помощью неё мы сможем построить график и наглядно показать, какой из параметров вносит наибольший вклад
- **Вычислительные ресурсы**

Поскольку обученные модели будут использоваться внутри рекомендателя, хотелось бы иметь хорошую скорость на этапе предсказания

 - Линейная регрессия выполняет только операцию скалярного произведения, что работает мгновенно
 - Обученный Random Forest тратит время только на проход по веткам деревьев фиксированной глубины, что также имеет константную сложность $O(1)$ относительно объема данных

Анализ корреляции между метриками

Анализ на объединённых данных всех подходов проверяет характер связей между метриками: являются ли они линейными или нелинейными монотонными.

Диагностика: Пирсон vs Спирмен

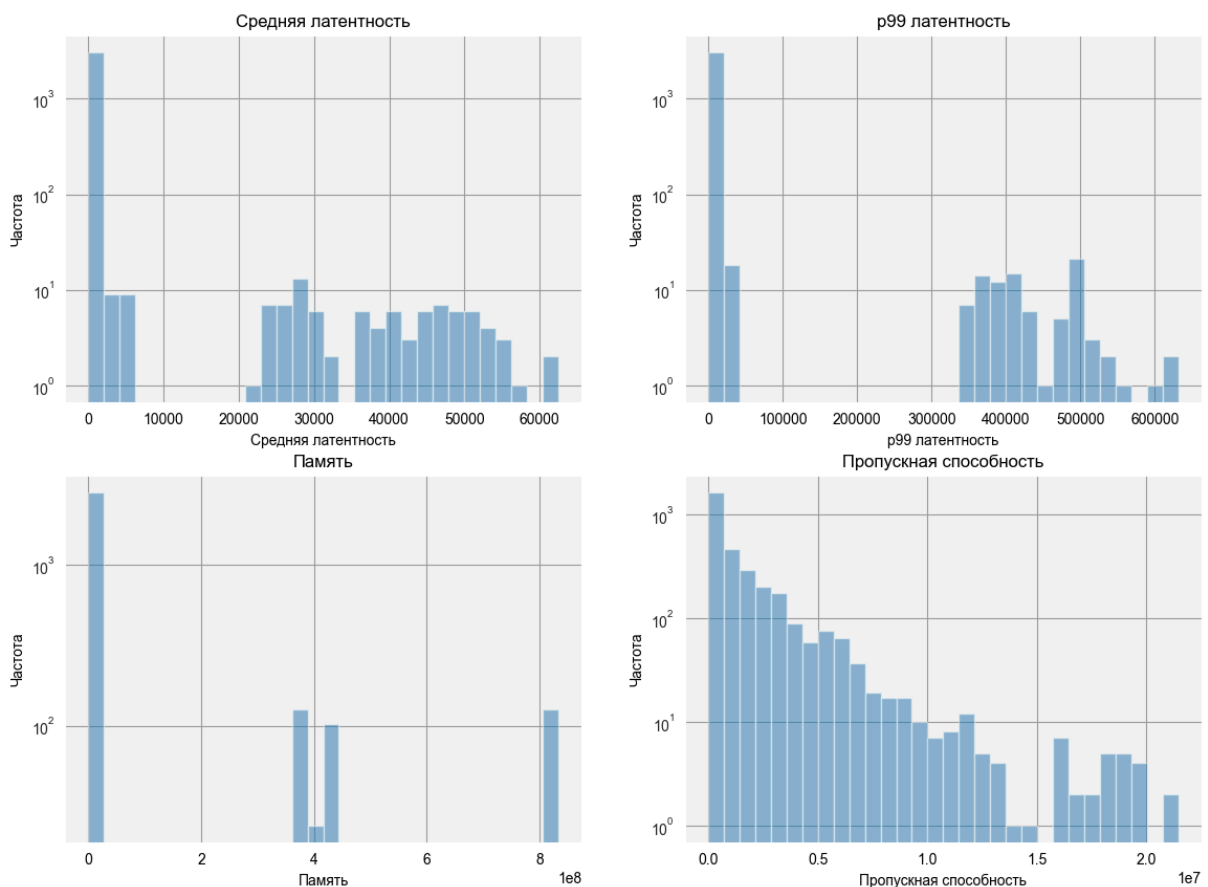
Пара метрик	Pearson	Spearman	Разрыв	R^2 лин.	R^2 квадр.
avg_latency vs p99_latency	0.953	0.971	0.018	0.908	0.960
avg_latency vs memory	0.321	0.499	0.178	0.103	0.114
avg_latency vs throughput	-0.111	-0.998	0.887	0.012	0.015
p99_latency vs memory	0.327	0.472	0.144	0.107	0.112
p99_latency vs throughput	-0.113	-0.971	0.858	0.013	0.015
memory vs throughput	-0.169	-0.485	0.316	0.028	0.036

Анализ ключевых паттернов

- **Латентность vs Throughput (критическая нелинейность):** Наблюдается экстремальный разрыв между Пирсоном (≈ -0.11) и Спирменом (≈ -0.99). Связь почти детерминированная, но обратно пропорциональная. Линейная модель (Ridge) принимает эту гиперболу за случайный шум, чем объясняется её провал на throughput ($R^2 < 0.58$). Низкий квадратичный R^2 (0.015) доказывает неэффективность простых полиномов для аппроксимации этой формы связи.
- **avg_latency vs p99_latency (линейный полюс):** Разрыв минимален (0.018), линейная модель сразу объясняет 90.8% дисперсии. Метрики тесно связаны линейно, а их совместные отклонения вызваны внешними факторами (пиковыми нагрузками), а не формой взаимозависимости.
- **Память (структурная автономность):** Связь метогу с обеими латентностями слабая в любой форме (Spearman $\approx 0.47 - 0.50$). Это подтверждает, что объём структуры определяется преимущественно статическими геометрическими параметрами, а не временной динамикой выполнения операций.

Анализ многомерных графиков распределений

Распределение метрик



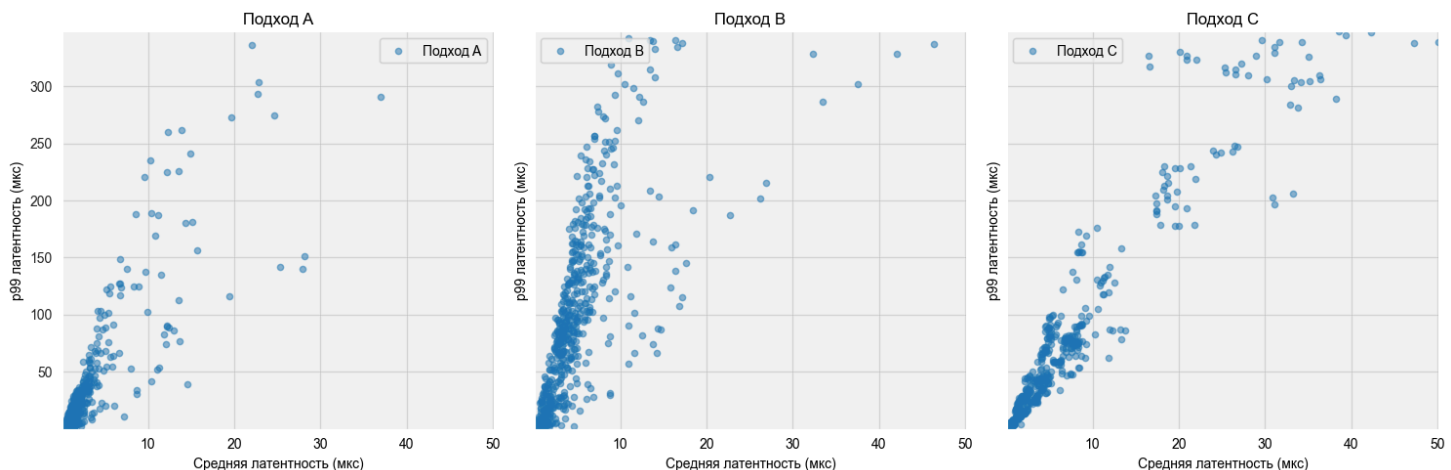
Средняя латентность и p99 демонстрируют схожую бимодальную структуру: основная масса наблюдений сосредоточена вблизи нуля, а разреженный хвост тянется до больших значений.

Память имеет три выраженные полосы, что отражает дискретность сетки параметров: объём памяти определяется числом узлов и меняется скачкообразно при смене D и W.

Пропускная способность убывает монотонно без явных пробелов - следствие нелинейной связи throughput, которая сглаживает бимодальность латентности.

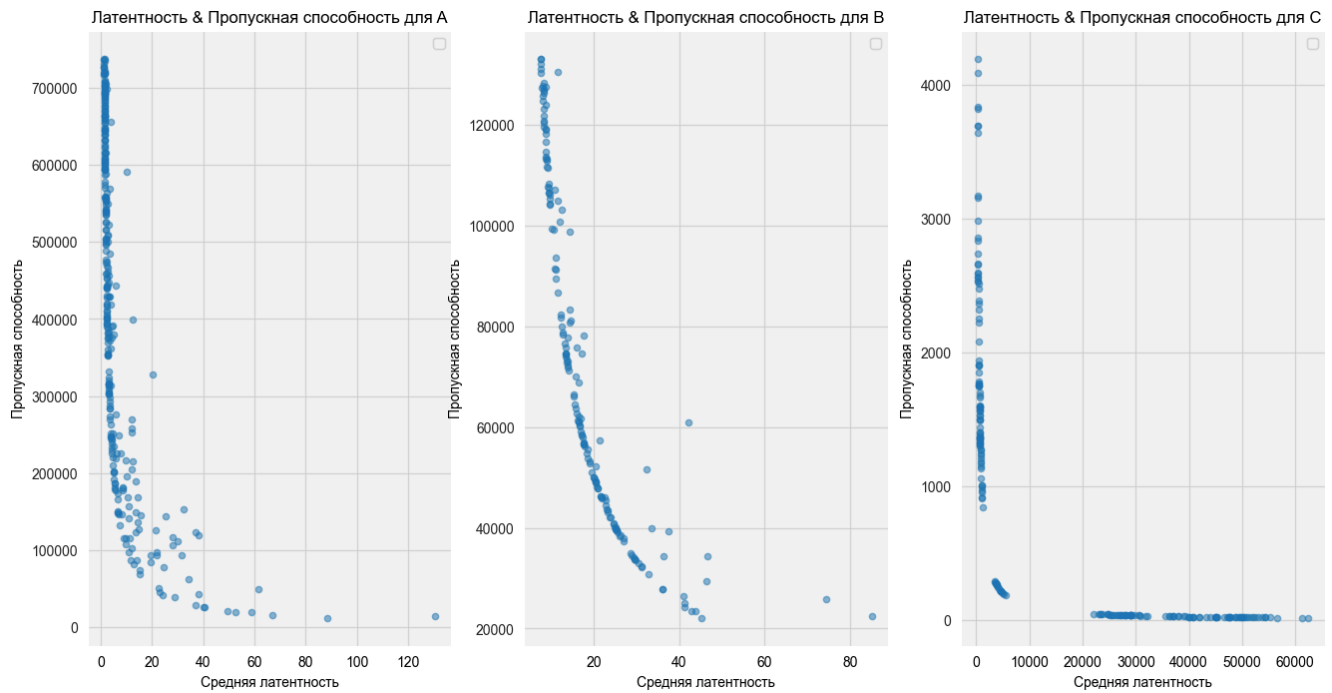
Анализ совместного распределения задержек (p99 vs avg_latency)

Для детального изучения структуры задержек в области стабильной работы системы (с отсечением экстремальных выбросов по порогу $p99_latency_us < 350$ мкс) были проанализированы разделенные графики рассеяния с единым масштабом осей для подходов A, B и C



1. **Подход А (Наивное дерево)** Большинство точек сгруппировано в безопасной зоне низких задержек (до 5 мкс по $avg_latency$ и 50 мкс по $p99$). Однако при усложнении поиска линейная зависимость теряется, и график начинает сильно ветвиться. При росте средней задержки всего до 10–15 мкс происходят резкие скачки $p99$ вверх (до 250+ мкс). Это показывает, что наивное дерево теряет стабильность на тяжелых операциях, выдавая непредсказуемые выбросы
2. **Подход В (Гибрид с хэш-таблицей)** График вытянут в строгий вертикальный столб: средняя задержка намертво зажата в узком диапазоне (до 5–7 мкс), но разброс $p99$ огромен (до 340+ мкс). За счет поиска за $O(1)$ архитектура отлично сдерживает рост среднего времени ответа. Платой за это становятся редкие, но тяжелые операции (коллизии, рехэширование или инвалидация кэша), которые вызывают огромные всплески хвостовой латентности
3. **Подход С (хэш-таблица)** Здесь видна четкая ступенчатая зависимость - точки собираются в изолированные «островки» в районах 7, 18 и 33 мкс по оси X. Подход деградирует предсказуемо: $p99$ растет строго пропорционально средней задержке. Разделение на кластеры указывает на явные шаги в архитектуре (например, переход на новый уровень вложенности или чтение глубоких поддиректорий)

Плотности распределения пропускной способности throughput_ops_sec

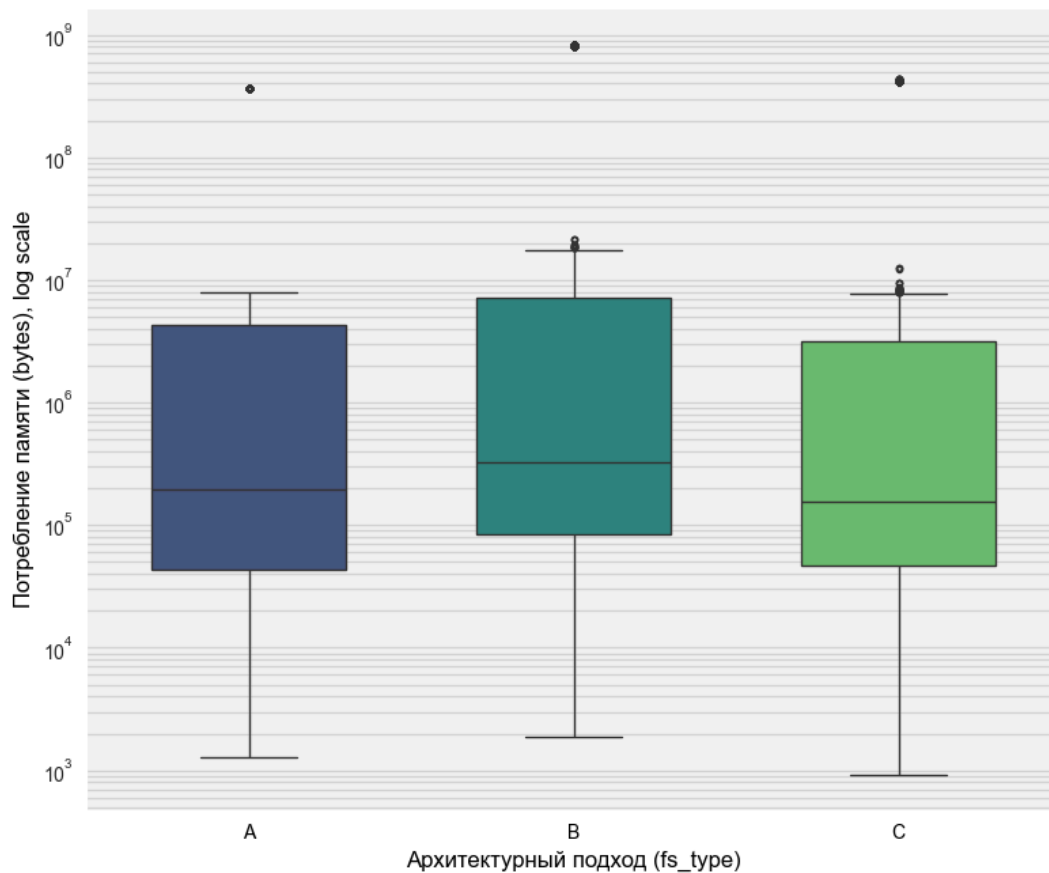


На рисунке представлены три scatter plot-а, отображающих зависимость пропускной способности от средней латентности для трёх реализаций.

Заметно влияние деревьев в реализации, при приближении к началу координат увеличивается рассеивание точек.

Исследование профилей памяти

Распределение потребления памяти в зависимости от подхода и фактора заполнения (F)



Подход В демонстрирует самые высокие показатели потребления памяти по всем ключевым метрикам: у него наибольшее медианное значение (более $3 \cdot 10^5$ байт), самый высокий верхний квартиль, а также максимальный среди всех групп единичный выброс, достигающий почти 10^9 байт

Подходы А и С имеют схожие минимальные значения (около 10^3 байт) и сопоставимые нижние границы распределения. Однако подход А имеет более высокую медиану и более широкий межквартильный размах по сравнению с подходом С, что говорит о стабильно большем потреблении памяти в типичных случаях

Подход С выглядит наиболее эффективным с точки зрения экономии ресурсов, так как его медиана находится на самом низком уровне (примерно $1.5 \cdot 10^5$ байт), а основная масса данных сдвинута ниже, чем у остальных подходов. При этом во всех трех архитектурных решениях наблюдаются единичные аномальные выбросы в районе 10^8 – 10^9 байт, что указывает на наличие специфических ресурсоемких сценариев для каждого из вариантов

Оценки моделей

Оценка модели для А:

Метрика	R^2 Ridge	R^2 RF	MAE RF	RMSE RF	MAPE RF
memory_usage_bytes	0.753	1.0000	145468	324847	63.71 %
avg_latency_us	0.270	0.7788	0.96	3.62	46.98 %

p99_latency_us	0.326	0.6968	14.83	51.77	140.72 %
throughput_ops_sec	0.413	0.6578	470530	802484	37.52 %

Метрики качества временных параметров оказались ниже, чем у остальных подходов. Стоит отметить, что интегральные оценки для модели А занижены из-за 1% экспериментальных точек, имеющих экстремально большие значения латентности. Модель плохо уловила тенденцию этого ультра-хвостового распределения в зонах аномальных всплесков. При этом на остальных 99% данных модель демонстрирует стабильную работу и нормальную предсказательную способность, адекватно описывая основную массу распределения.

Оценка модели для В:

Метрика	R^2 Ridge	R^2 RF	MAE RF	RMSE RF	MAPE RF
memory_usage_bytes	0.7459	0.9996	1 618 333	5 450 234	181.09 %
avg_latency_us	0.6034	0.8399	1.27	3.07	32.87 %
p99_latency_us	0.6735	0.8386	27.33	66.27	54.18 %
throughput_ops_sec	0.5538	0.9100	377 639	776 423	25.85 %

Оценка модели для С:

Метрика	R^2 Ridge	R^2 RF	MAE RF	RMSE RF	MAPE RF
memory_usage_bytes	0.6134	0.9988	1 313 819	4 743 374	101.01 %
avg_latency_us	0.4653	0.9859	338.82	1 402.37	2533.07 %
p99_latency_us	0.4749	0.9290	8 797.01	32 839.16	2595.51 %
throughput_ops_sec	0.4672	0.8808	627 442	1 547 501	579.56 %

Вывод: анализ стратегий выбора подходов (А, В, С)

Общие закономерности

При согласовании всех критериев выбор подхода определяется геометрией дерева:

Область параметров	Подход	Примеры профилей нагрузки
Очень малые деревья ($D \leq 2, W \leq 10$)	С	backup, web_server, database
Малые деревья, много ветвей ($D = 2, W \geq 15$)	В (редко А)	backup, web_server
Средние деревья ($3 \leq D \leq 10, W \leq 15$)	А	build_system, file_manager, refactoring, reshaping
Большие деревья ($D \geq 15$ или $W \geq 30$)	В (иногда А)	backup, web_server, большинство профилей при $W = 30 - 50$

Специфичные для баз данных ($D \leq 5, W \leq 15$) С database (стабильный приоритет С)

Роль подходов в зависимости от структуры:

- **С** — доминирует на малых глубинах ($D \leq 5$), а для профиля database остается основным выбором вплоть до средних размеров ($D \leq 5, W \leq 15$).
- **А** — универсальный выбор для средних и крупных деревьев ($D \geq 3, W \leq 15$) в большинстве системных профилей.
- **В** — преобладает при экстремальном ветвлении ($W \geq 30$) либо большой глубине ($D \geq 15$).

Влияние профиля нагрузки (profile_name)

- **build_system, file_manager, refactoring, reshaping:** почти всегда выбирают **А**, переключаясь на **В** только при критическом расширении ширины ($W \geq 30$).
- **backup, web_server:** адаптивны - предпочитают **С** для мелких деревьев, **В** при $D = 2, W \geq 15$, и переходят к **А** на средних масштабах.
- **database:** демонстрирует жесткую специфику. Подход **С** эффективен вплоть до $D = 5$. Смена на **А** происходит только при $D = 10, W \geq 10$. Подход **В** практически не востребован.

Влияние стратегии взвешивания

- **memory:** тяготеет к **А**, что указывает на оптимальное использование памяти в этом подходе, так же встречается **С** на небольших деревьях.
- **throughput:** чаще других выбирает **С** (для малых деревьев) или **В** (для крупных), обеспечивая пиковую пропускную способность там, где сбалансированная стратегия выбирает **А**.
- **latency** схож с дефолтным **balanced**, но в пограничных ситуациях склоняются к **А** из-за более стабильного времени отклика.

Рекомендации по выбору

Приоритет / Условие	Подход	Контекст применения
Очень малые деревья или СУБД ($D \leq 5$)	С	Оптимально для database, web_server, backup
Сбалансированная производительность	А	Основная масса сценариев ($D \geq 3, W \leq 15$)
Строгая минимизация памяти	А	Эффективен во всех профилях
Максимизация пропускной способности	В / С	С - для малых структур/СУБД В - при ($W \geq 30$ или $D \geq 15$)
Минимизация времени	А / В	А - для средних деревьев; В - для тяжелых/широких

- **Подход А** - наиболее универсальный компромисс, идеален для экономии памяти и сбалансированных нагрузок на средних/крупных структурах.

- **Подход В** - незаменим для сверхглубоких или сильно разветвленных деревьев, где важна сырая пропускная способность.
- **Подход С** - целевой и высокоэффективный выбор для небольших деревьев.